EXPERIMENT-1:

Implement multilayer perceptron algorithm for MNIST Hand written Digit Classification.

Aim: Implement multilayer perceptron algorithm for MNIST Hand written Digit Classification.

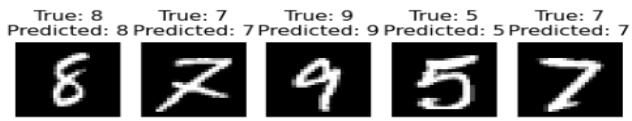
PROGRAM:

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
import numpy as npS
import matplotlib.pyplot as plt
# Load and preprocess the MNIST dataset
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
train_images, test_images = train_images / 255.0, test_images / 255.0
# Flatten the images and one-hot encode the labels
train_images = train_images.reshape((60000, 28 * 28))
test_images = test_images.reshape((10000, 28 * 28))
train_labels = tf.keras.utils.to_categorical(train_labels)
test_labels = tf.keras.utils.to_categorical(test_labels)
# Define the MLP model
model = models.Sequential()
model.add(layers.Dense(128, activation='relu', input_shape=(28 * 28,)))
model.add(layers.Dropout(0.2))
model.add(layers.Dense(10, activation='softmax'))
# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
# Train the model
```

```
model.fit(train_images, train_labels, epochs=5, batch_size=64, validation_split=0.2)
# Evaluate the model on the test set
test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f'Test accuracy: {test_acc}')
# Make predictions on some test samples
num_samples = 5
sample_indices = np.random.choice(test_images.shape[0], num_samples, replace=False)
samples_to_predict = test_images[sample_indices]
predictions = model.predict(samples_to_predict)
# Display the predictions with images
for i in range(num_samples):
    predicted_label = np.argmax(predictions[i])
    true_label = np.argmax(test_labels[sample_indices[i]])
    # Reshape the flattened image back to (28, 28)
    sample_image = samples_to_predict[i].reshape((28, 28))
    plt.subplot(1, num_samples, i + 1)
    plt.imshow(sample_image, cmap='gray')
    plt.title(f"True: {true_label}\nPredicted: {predicted_label}")
    plt.axis('off')
plt.show()
```

OUTPUT:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 [==============================] - 29s 3us/step
Epoch 1/5
750/750 [==============================] - 3s 3ms/step - loss: 0.3725 - accuracy: 0.8942 - val_loss: 0.1870 - val_accuracy: 0.9
450
Epoch 2/5
750/750 [==============================] - 2s 3ms/step - loss: 0.1818 - accuracy: 0.9470 - val_loss: 0.1358 - val_accuracy: 0.9
595
Epoch 3/5
750/750 [==============================] - 2s 3ms/step - loss: 0.1376 - accuracy: 0.9598 - val_loss: 0.1132 - val_accuracy: 0.9
672
Epoch 4/5
750/750 [==============================] - 2s 3ms/step - loss: 0.1099 - accuracy: 0.9682 - val_loss: 0.0964 - val_accuracy: 0.9
717
Epoch 5/5
750/750 [==============================] - 2s 3ms/step - loss: 0.0942 - accuracy: 0.9718 - val_loss: 0.0932 - val_accuracy: 0.9
723
313/313 [==============================] - 1s 2ms/step - loss: 0.0853 - accuracy: 0.9733
Test accuracy: 0.9732999801635742
1/1 [==============================] - 0s 76ms/step
```



EXPERIMENT-2:

Design a neural network for classifying movie reviews (Binary Classification) using IMDB dataset.

Aim: Design a neural network for classifying movie reviews (Binary Classification) using IMDB dataset..

PROGRAM:

```
import tensorflow as tf

from tensorflow.keras.datasets import imdb

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Embedding, Flatten, Dense

from tensorflow.keras.preprocessing.sequence import pad_sequences

# Load the IMDb dataset and preprocess it

max_features, maxlen = 10000, 100

(train_data, train_labels), (test_data, test_labels) = imdb.load_data(num_words=max_features)

train_data, test_data = pad_sequences(train_data, maxlen=maxlen), pad_sequences(test_data,
maxlen=maxlen)

# Define, compile, and train the model

model = Sequential([

    Embedding(max_features, 8, input_length=maxlen),

    Flatten(),

    Dense(1, activation='sigmoid')

])

model.compile(optimizer='rmsprop', loss='binary_crossentropy', metrics=['accuracy'])

model.fit(train_data, train_labels, epochs=10, batch_size=32, validation_split=0.2)

# Evaluate and print the model's accuracy

print(f'Test Accuracy: {model.evaluate(test_data, test_labels)[1] * 100:.2f}%')

# Function to process and predict sentiment of reviews

word_index = imdb.get_word_index()
```

```
def predict_sentiment(review, word_index, model, maxlen):
    # Tokenize and pad the review
    tokens = pad_sequences([[word_index.get(word, 0) for word in review.lower().split()]],
maxlen=maxlen)
    prediction = model.predict(tokens)
    return prediction[0, 0]
# Predict sentiment for sample reviews
positive_review = "This movie was fantastic! The acting and plot were amazing."
negative_review = "I hated this movie. The acting was terrible, and the plot made no sense."
positive_prediction = predict_sentiment(positive_review, word_index, model, maxlen)
negative_prediction = predict_sentiment(negative_review, word_index, model, maxlen)
print(f'Positive Review Prediction: {positive_prediction:.4f}')
print(f'Negative Review Prediction: {negative_prediction:.4f}')
```

OUTPUT:

```
Epoch 1/10
625/625 [==============================] - 2s 2ms/step - loss: 0.6336 - accuracy: 0.6665 - val_loss: 0.5008 - val_accuracy: 0.7
906
Epoch 2/10
625/625 [==============================] - 1s 2ms/step - loss: 0.3918 - accuracy: 0.8431 - val_loss: 0.3627 - val_accuracy: 0.8
418
Epoch 3/10
625/625 [==============================] - 1s 2ms/step - loss: 0.2985 - accuracy: 0.8781 - val_loss: 0.3358 - val_accuracy: 0.8
520
Epoch 4/10
625/625 [==============================] - 1s 2ms/step - loss: 0.2554 - accuracy: 0.8972 - val_loss: 0.3306 - val_accuracy: 0.8
584
Epoch 5/10
625/625 [==============================] - 1s 2ms/step - loss: 0.2251 - accuracy: 0.9122 - val_loss: 0.3302 - val_accuracy: 0.8
598
Epoch 6/10
625/625 [==============================] - 1s 2ms/step - loss: 0.2002 - accuracy: 0.9244 - val_loss: 0.3381 - val_accuracy: 0.8
550
Epoch 7/10
625/625 [==============================] - 1s 2ms/step - loss: 0.1782 - accuracy: 0.9327 - val_loss: 0.3445 - val_accuracy: 0.8
550
Epoch 8/10
625/625 [==============================] - 1s 2ms/step - loss: 0.1583 - accuracy: 0.9423 - val_loss: 0.3568 - val_accuracy: 0.8
506
Epoch 9/10
625/625 [==============================] - 1s 2ms/step - loss: 0.1397 - accuracy: 0.9510 - val_loss: 0.3767 - val_accuracy: 0.8
432
Epoch 10/10
625/625 [==============================] - 1s 2ms/step - loss: 0.1231 - accuracy: 0.9582 - val_loss: 0.3828 - val_accuracy: 0.8
426
782/782 [==============================] - 1s 1ms/step - loss: 0.3728 - accuracy: 0.8485
Test Accuracy: 84.85%
WARNING:tensorflow:5 out of the last 77 calls to <function Model.make_predict_function.<locals>.predict_function at 0x000001FD3
02E4040> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creatin
g @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors.
For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can a
void unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and http
s://www.tensorflow.org/api_docs/python/tf/function for  more details.
1/1 [==============================] - 0s 44ms/step
1/1 [==============================] - 0s 22ms/step
Positive Review Prediction: 0.2941
Negative Review Prediction: 0.5009
```

EXPERIMENT-3:

Design a neural Network for classifying news wires (Multi class classification) using Reuters dataset.

Aim: Design a neural Network for classifying news wires (Multi class classification) using Reuters dataset…

PROGRAM:

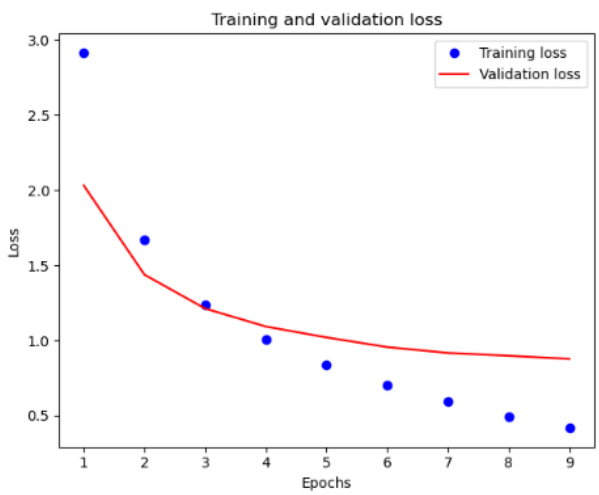
```
import numpy as np
import tensorflow as tf
from tensorflow.keras.datasets import reuters
from tensorflow.keras import models, layers
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
# Load the dataset
(train_data, train_labels), (test_data, test_labels) = reuters.load_data(num_words=10000)
# Function to vectorize sequences
def vectorize_sequences(sequences, dimension=10000):
    results = np.zeros((len(sequences), dimension))
    for i, sequence in enumerate(sequences):
        results[i, sequence] = 1.
    return results
# Vectorize data
x_train = vectorize_sequences(train_data)
x_test = vectorize_sequences(test_data)
# One-hot encode labels
one_hot_train_labels = to_categorical(train_labels)
one_hot_test_labels = to_categorical(test_labels)
# Reserve validation set
x_val = x_train[:1000]
```

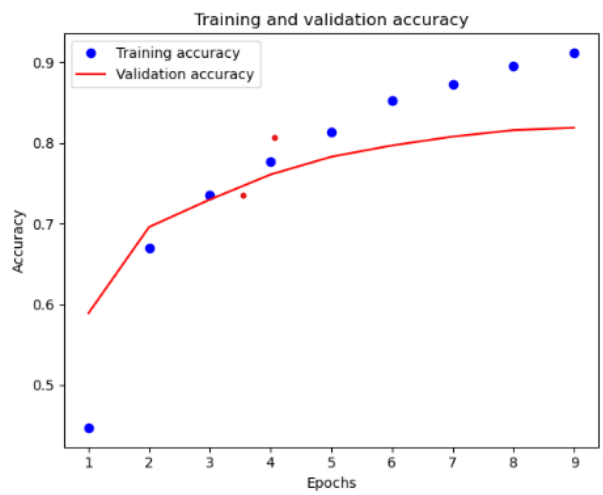
```
partial_x_train = x_train[1000:]
y_val = one_hot_train_labels[:1000]
partial_y_train = one_hot_train_labels[1000:]
# Define and compile the model
model = models.Sequential([
    layers.Dense(64, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(46, activation='softmax')
])
model.compile(optimizer='rmsprop', loss='categorical_crossentropy', metrics=['accuracy'])
# Train the model
history = model.fit(partial_x_train, partial_y_train, epochs=9,
                    batch_size=512, validation_data=(x_val, y_val))
# Function to plot the training history for loss and accuracy
def plot_history(history):
    acc = history.history['accuracy']
    val_acc = history.history['val_accuracy']
    loss = history.history['loss']
    val_loss = history.history['val_loss']
    epochs = range(1, len(acc) + 1)
    # Plot training and validation loss
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)  # 1 row, 2 columns, first plot
    plt.plot(epochs, loss, 'bo', label='Training loss')
    plt.plot(epochs, val_loss, 'r', label='Validation loss')
    plt.title('Training and validation loss')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
```

```
    # Plot training and validation accuracy
    plt.subplot(1, 2, 2)  # 1 row, 2 columns, second plot
    plt.plot(epochs, acc, 'bo', label='Training accuracy')
    plt.plot(epochs, val_acc, 'r', label='Validation accuracy')
    plt.title('Training and validation accuracy')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.tight_layout()  # Adjusts the plot to ensure everything fits without overlapping
    plt.show()
# Plot the training and validation loss and accuracy
plot_history(history)
# Evaluate the model on test data
results = model.evaluate(x_test, one_hot_test_labels)
print(f"Test Loss: {results[0]}")
print(f"Test Accuracy: {results[1]}")
# Compare against a random baseline
test_labels_copy = test_labels.copy()
np.random.shuffle(test_labels_copy)
baseline_accuracy = np.sum(np.array(test_labels) == np.array(test_labels_copy)) /
len(test_labels)
print(f"Random Baseline Accuracy: {baseline_accuracy}")
# Make predictions
predictions = model.predict(x_test)
```

OUTPUT:

```
Epoch 1/9
16/16 [==============================] - 1s 39ms/step - loss: 2.9159 - accuracy: 0.4468 - val_loss: 2.0314 - val_accuracy: 0.58
90
Epoch 2/9
16/16 [==============================] - 0s 14ms/step - loss: 1.6673 - accuracy: 0.6699 - val_loss: 1.4377 - val_accuracy: 0.69
60
Epoch 3/9
16/16 [==============================] - 0s 14ms/step - loss: 1.2377 - accuracy: 0.7355 - val_loss: 1.2122 - val_accuracy: 0.73
00
Epoch 4/9
16/16 [==============================] - 0s 14ms/step - loss: 1.0048 - accuracy: 0.7767 - val_loss: 1.0926 - val_accuracy: 0.76
10
Epoch 5/9
16/16 [==============================] - 0s 14ms/step - loss: 0.8402 - accuracy: 0.8135 - val_loss: 1.0200 - val_accuracy: 0.78
30
Epoch 6/9
16/16 [==============================] - 0s 14ms/step - loss: 0.7023 - accuracy: 0.8524 - val_loss: 0.9558 - val_accuracy: 0.79
70
Epoch 7/9
16/16 [==============================] - 0s 15ms/step - loss: 0.5931 - accuracy: 0.8723 - val_loss: 0.9169 - val_accuracy: 0.80
80
Epoch 8/9
16/16 [==============================] - 0s 15ms/step - loss: 0.4956 - accuracy: 0.8961 - val_loss: 0.8981 - val_accuracy: 0.81
60
Epoch 9/9
16/16 [==============================] - 0s 13ms/step - loss: 0.4167 - accuracy: 0.9122 - val_loss: 0.8771 - val_accuracy: 0.81
90
```






```
71/71 [==============================] - 0s 2ms/step - loss: 0.9679 - accuracy: 0.7867
Test Loss: 0.9678676724433899
Test Accuracy: 0.7867319583892822
Random Baseline Accuracy: 0.19679430097951914
71/71 [==============================] - 0s 2ms/step
```

EXPERIMENT- 4:

Design a neural network for predicting house prices using Boston Housing Price dataset.

Aim: Design a neural network for predicting house prices using Boston Housing Price dataset….

PROGRAM:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
# Load data from CSV file
data = pd.read_csv('C:\\Users\\KOTES\\Downloads\\HousingData.csv')
# Separate features and target variable
X = data.drop('MEDV', axis=1).values
y = data['MEDV'].values
# Normalize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
# Define the model architecture
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(64, activation='relu'),
```

```
    Dense(1)  # Output layer with linear activation

])
# Compile the model
model.compile(optimizer='adam', loss='mean_squared_error')
# Train the model
history = model.fit(X_train, y_train, epochs=100, batch_size=32, validation_split=0.1)
# Evaluate the model
test_loss = model.evaluate(X_test, y_test)
print(f'Test Loss: {test_loss}')
# Predict house prices
y_pred = model.predict(X_test)S
# Visualize predictions vs actual values
plt.scatter(y_test, y_pred)
plt.xlabel('True Prices')
plt.ylabel('Predicted Prices')
plt.title('Predicted Prices vs True Prices')
plt.show()
```

OUTPUT:

```
Epoch 1/100
12/12 [==============================] - 1s 12ms/step - loss: 605.5467 - val_loss: 596.9451
Epoch 2/100
12/12 [==============================] - 0s 4ms/step - loss: 602.5436 - val_loss: 593.3427
Epoch 3/100
12/12 [==============================] - 0s 5ms/step - loss: 598.8230 - val_loss: 589.2816
Epoch 4/100
12/12 [==============================] - 0s 5ms/step - loss: 594.5688 - val_loss: 584.6063
Epoch 5/100
12/12 [==============================] - 0s 4ms/step - loss: 589.5783 - val_loss: 579.2087
Epoch 6/100
12/12 [==============================] - 0s 5ms/step - loss: 583.8889 - val_loss: 572.9472
Epoch 7/100
12/12 [==============================] - 0s 5ms/step - loss: 577.3969 - val_loss: 565.9156
Epoch 8/100
12/12 [==============================] - 0s 5ms/step - loss: 570.0383 - val_loss: 558.0862
Epoch 9/100
12/12 [==============================] - 0s 4ms/step - loss: 561.9127 - val_loss: 549.3073
```

```
Epoch 93/100
12/12 [==============================] - 0s 4ms/step - loss: 88.5479 - val_loss: 72.1618
Epoch 94/100
12/12 [==============================] - 0s 5ms/step - loss: 88.5500 - val_loss: 72.1504
Epoch 95/100
12/12 [==============================] - 0s 4ms/step - loss: 88.5475 - val_loss: 72.1479
Epoch 96/100
12/12 [==============================] - 0s 4ms/step - loss: 88.5678 - val_loss: 72.1315
Epoch 97/100
12/12 [==============================] - 0s 4ms/step - loss: 88.5468 - val_loss: 72.1332
Epoch 98/100
12/12 [==============================] - 0s 4ms/step - loss: 88.5554 - val_loss: 72.1445
Epoch 99/100
12/12 [==============================] - 0s 4ms/step - loss: 88.5453 - val_loss: 72.1454
Epoch 100/100
12/12 [==============================] - 0s 4ms/step - loss: 88.5415 - val_loss: 72.1415
4/4 [==============================] - 0s 0s/step - loss: 74.9023
Test Loss: 74.90230560302734
4/4 [==============================] - 0s 2ms/step
```

EXPERIMENT- 5:

Build a Convolution Neural Network for MNIST Hand written Digit Classification.

Aim: Build a Convolution Neural Network for MNIST Hand written Digit Classification.

PROGRAM:

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
# Load and preprocess the MNIST dataset
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
train_images = train_images.reshape((60000, 28, 28, 1)).astype('float32') / 255
test_images = test_images.reshape((10000, 28, 28, 1)).astype('float32') / 255
# Define the CNN model with reduced complexity
model = models.Sequential([
layers.Conv2D(16, (3, 3), activation='relu', input_shape=(28, 28, 1)),
layers.MaxPooling2D((2, 2)),
layers.Conv2D(32, (3, 3), activation='relu'),
layers.MaxPooling2D((2, 2)),
layers.Flatten(),
layers.Dense(32, activation='relu'),
layers.Dense(10, activation='softmax')
])
# Compile the model
model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
# Define early stopping callback
```

```
early_stopping = tf.keras.callbacks.EarlyStopping(monitor='val_loss',
patience=2)
# Train the model with increased batch size and early stopping
model.fit(train_images, train_labels, epochs=20, batch_size=128,
validation_split=0.2, callbacks=[early_stopping])
# Evaluate the model
test_loss, test_acc = model.evaluate(test_images, test_labels)
print('Test accuracy:', test_acc)
```

OUTPUT:

```
WARNING:tensorflow:From C:\Users\KOTES\anaconda3\lib\site-packages\keras\src\layers\pooling\max_pooling2d.py:161: The name t
f.nn.max_pool is deprecated. Please use tf.nn.max_pool2d instead.

WARNING:tensorflow:From C:\Users\KOTES\anaconda3\lib\site-packages\keras\src\optimizers\__init__.py:309: The name tf.train.O
ptimizer is deprecated. Please use tf.compat.v1.train.Optimizer instead.

Epoch 1/20
WARNING:tensorflow:From C:\Users\KOTES\anaconda3\lib\site-packages\keras\src\utils\tf_utils.py:492: The name tf.ragged.Ragge
dTensorValue is deprecated. Please use tf.compat.v1.ragged.RaggedTensorValue instead.

WARNING:tensorflow:From C:\Users\KOTES\anaconda3\lib\site-packages\keras\src\engine\base_layer_utils.py:384: The name tf.exe
cuting_eagerly_outside_functions is deprecated. Please use tf.compat.v1.executing_eagerly_outside_functions instead.

375/375 [==============================] - 4s 8ms/step - loss: 0.3496 - accuracy: 0.9011 - val_loss: 0.1225 - val_accuracy:
0.9651
Epoch 2/20
375/375 [==============================] - 3s 7ms/step - loss: 0.0989 - accuracy: 0.9691 - val_loss: 0.0795 - val_accuracy:
0.9759
Epoch 3/20
375/375 [==============================] - 3s 7ms/step - loss: 0.0721 - accuracy: 0.9774 - val_loss: 0.0616 - val_accuracy:
0.9822
Epoch 4/20
375/375 [==============================] - 3s 8ms/step - loss: 0.0568 - accuracy: 0.9819 - val_loss: 0.0575 - val_accuracy:
0.9822
Epoch 5/20
375/375 [==============================] - 3s 8ms/step - loss: 0.0480 - accuracy: 0.9854 - val_loss: 0.0666 - val_accuracy:
0.9806
Epoch 6/20
375/375 [==============================] - 3s 8ms/step - loss: 0.0442 - accuracy: 0.9865 - val_loss: 0.0616 - val_accuracy:
0.9818
313/313 [==============================] - 1s 2ms/step - loss: 0.0525 - accuracy: 0.9827
Test accuracy: 0.982699990272522
```

EXPERIMENT- 6:

Build a Convolution Neural Network for simple image (dogs and Cats) Classification

Aim: Build a Convolution Neural Network for simple image (dogs and Cats) Classification

PROGRAM:

```
import tensorflow as tf
from tensorflow import keras
from keras import Sequential
from keras.layers import Dense, Conv2D, MaxPooling2D, Flatten
# Generators
# Define the training dataset using image_dataset_from_directory
train_data = keras.utils.image_dataset_from_directory(
    directory="C:\\Users\\KOTES\\OneDrive\\Desktop\\cat & dog\\training_set",
    labels='inferred',
    label_mode='int',
    batch_size=32,
    image_size=(256, 256)
)
# Define the validation dataset using image_dataset_from_directory
validation_data = keras.utils.image_dataset_from_directory(
    directory="C:\\Users\\KOTES\\OneDrive\\Desktop\\cat & dog\\test_set",
    labels='inferred',
    label_mode='int',
    batch_size=32,
    image_size=(256, 256)
)
```

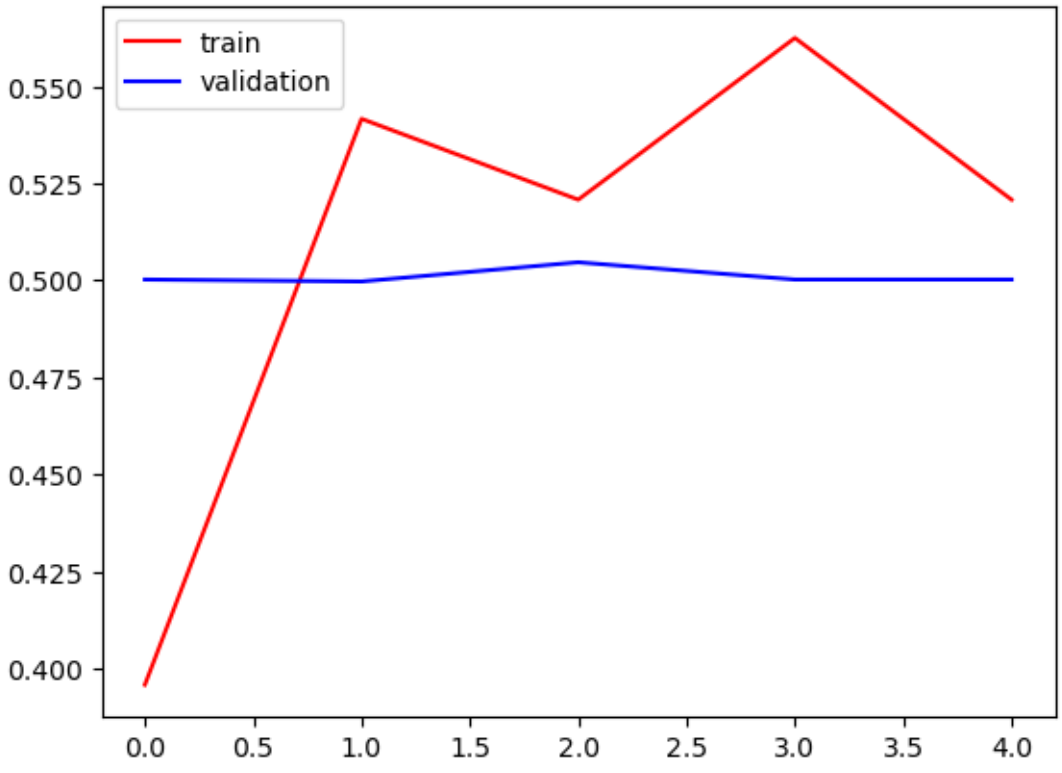
```
# Normalize the image data
def process(image, label):
    image = tf.cast(image/255, tf.float32)
    return image, label
train_data = train_data.map(process)
validation_data = validation_data.map(process)
# Create CNN Model
model = Sequential()
model.add(Conv2D(32, kernel_size=(3, 3), padding='valid', activation='relu', input_shape=(256,
256, 3)))
model.add(MaxPooling2D(pool_size=(2, 2), strides=2, padding='valid'))
model.add(Conv2D(64, kernel_size=(3, 3), padding='valid', activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2), strides=2, padding='valid'))
model.add(Conv2D(128, kernel_size=(3, 3), padding='valid', activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2), strides=2, padding='valid'))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(64, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.summary()
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
# Define the number of epochs
epochs = 5  # Adjust this value based on your training requirements
# Train the model using fit_generator
history = model.fit_generator(
    train_data,
    validation_data=validation_data,
    steps_per_epoch=3,  # Adjust this value based on your dataset size and batch size
```

```
    epochs=epochs,  # Adjust this value based on your validation dataset size and batch size
    verbose=1
)
# Plot training and validation accuracy
import matplotlib.pyplot as plt
plt.plot(history.history['accuracy'], color='red', label='train')
plt.plot(history.history['val_accuracy'], color='blue', label='validation')
plt.legend()
plt.show()
```

OUTPUT:

```
Found 8005 files belonging to 2 classes.
Found 2023 files belonging to 2 classes.
WARNING:tensorflow:From C:\Users\KOTES\anaconda3\lib\site-packages\keras\src\layers\pooling\max_pooling2d.py:161: The name t
f.nn.max_pool is deprecated. Please use tf.nn.max_pool2d instead.

Model: "sequential_2"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 254, 254, 32)      896

 max_pooling2d (MaxPooling2  (None, 127, 127, 32)      0
 D)

 conv2d_1 (Conv2D)           (None, 125, 125, 64)      18496

 max_pooling2d_1 (MaxPoolin  (None, 62, 62, 64)        0
 g2D)

 conv2d_2 (Conv2D)           (None, 60, 60, 128)       73856

 max_pooling2d_2 (MaxPoolin  (None, 30, 30, 128)       0
 g2D)

 flatten (Flatten)           (None, 115200)            0

 dense_2 (Dense)             (None, 128)               14745728

 dense_3 (Dense)             (None, 64)                8256

 dense_4 (Dense)             (None, 1)                 65
```

```
=================================================================
Total params: 14847297 (56.64 MB)
Trainable params: 14847297 (56.64 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
Epoch 1/5
C:\Users\KOTES\AppData\Local\Temp\ipykernel_5616\4047001894.py:46: UserWarning: `Model.fit_generator` is deprecated and will
be removed in a future version. Please use `Model.fit`, which supports generators.
  history = model.fit_generator(
3/3 [==============================] - 25s 10s/step - loss: 1.6404 - accuracy: 0.3958 - val_loss: 0.6931 - val_accuracy: 0.5
002
Epoch 2/5
3/3 [==============================] - 20s 10s/step - loss: 0.6941 - accuracy: 0.5417 - val_loss: 0.7010 - val_accuracy: 0.4
998
Epoch 3/5
3/3 [==============================] - 18s 9s/step - loss: 0.6968 - accuracy: 0.5208 - val_loss: 0.6907 - val_accuracy: 0.50
47
Epoch 4/5
3/3 [==============================] - 17s 8s/step - loss: 0.6791 - accuracy: 0.5625 - val_loss: 0.7053 - val_accuracy: 0.50
02
Epoch 5/5
3/3 [==============================] - 17s 8s/step - loss: 0.6907 - accuracy: 0.5208 - val_loss: 0.6924 - val_accuracy: 0.50
02
```



EXPERIMENT- 7:

Use a pre-trained convolution neural network (VGG 16) for image classification.

Aim: Use a pre-trained convolution neural network (VGG 16) for image classification.

PROGRAM:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt
from PIL import Image
# Load pre-trained VGG16 model
model = VGG16(weights='imagenet')
# Function to preprocess the image for VGG16
def preprocess_image(image_path):
 img = image.load_img(image_path, target_size=(224, 224))
 img_array = image.img_to_array(img)
 img_array = np.expand_dims(img_array, axis=0)
 img_array = preprocess_input(img_array)
 return img_array
# Function to make predictions
def predict(image_path):
 img_array = preprocess_image(image_path)
 predictions = model.predict(img_array)
 decoded_predictions = decode_predictions(predictions, top=3)[0] # Get top 3 predictions
 for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
        print("{}. {}: {:.2f}%".format(i + 1, label, score * 100))
```

```
# Display the image
img = Image.open(image_path)
plt.imshow(img)
plt.axis('off')
plt.show()
# Allow user to select an image
image_path = input("Enter the path of the image: ")
predict(image_path)
```

OUTPUT:

```
Enter the path of the image: dog.jpeg
1/1 [==============================] - 0s 219ms/step
1. German_shepherd: 92.16%
2. malinois: 6.66%
3. kelpie: 0.23%
```

EXPERIMENT- 8:

Implement one hot encoding of words or characters.

Aim: Implement one hot encoding of words or characters.

PROGRAM:

```
import pandas as pd

import numpy as np

import tensorflow as tf

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Load the employee dataset

data = pd.read_csv('C:\\Users\\KOTES\\Downloads\\Employee.csv')

# Perform one-hot encoding for categorical features

categorical_cols = ['Education', 'City', 'PaymentTier', 'Gender']

data_encoded = pd.get_dummies(data, columns=categorical_cols, drop_first=True)

# Convert 'EverBenched' column to numeric

data_encoded['EverBenched'] = data_encoded['EverBenched'].apply(lambda x: 1 if x == 'Yes'
else 0)

# Split the data into features and target

X = data_encoded.drop('LeaveOrNot', axis=1)

y = data_encoded['LeaveOrNot']

# Normalize numerical features

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# Split the data into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

# Build a simple deep learning model

model = tf.keras.models.Sequential([

    tf.keras.layers.Dense(64, activation='relu', input_shape=(X_scaled.shape[1],)),
```

```
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
# Train the model
model.fit(X_train, y_train, epochs=10, batch_size=32, validation_data=(X_test, y_test))
# Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)
print(f'Loss: {loss}, Accuracy: {accuracy}')
```

OUTPUT:

```
WARNING:tensorflow:From C:\Users\KOTES\anaconda3\lib\site-packages\keras\src\optimizers\__init__.py:309: The name tf.train.O
ptimizer is deprecated. Please use tf.compat.v1.train.Optimizer instead.

Epoch 1/10
WARNING:tensorflow:From C:\Users\KOTES\anaconda3\lib\site-packages\keras\src\utils\tf_utils.py:492: The name tf.ragged.Ragge
dTensorValue is deprecated. Please use tf.compat.v1.ragged.RaggedTensorValue instead.

WARNING:tensorflow:From C:\Users\KOTES\anaconda3\lib\site-packages\keras\src\engine\base_layer_utils.py:384: The name tf.exe
cuting_eagerly_outside_functions is deprecated. Please use tf.compat.v1.executing_eagerly_outside_functions instead.

117/117 [==============================] - 4s 9ms/step - loss: 0.5654 - accuracy: 0.7163 - val_loss: 0.4881 - val_accuracy:
0.7562
Epoch 2/10
117/117 [==============================] - 1s 6ms/step - loss: 0.4982 - accuracy: 0.7724 - val_loss: 0.4446 - val_accuracy:
0.7948
Epoch 3/10
117/117 [==============================] - 1s 5ms/step - loss: 0.4699 - accuracy: 0.7915 - val_loss: 0.4283 - val_accuracy:
0.8195
Epoch 4/10
117/117 [==============================] - 1s 5ms/step - loss: 0.4514 - accuracy: 0.8149 - val_loss: 0.4054 - val_accuracy:
0.8432
Epoch 5/10
117/117 [==============================] - 1s 5ms/step - loss: 0.4369 - accuracy: 0.8229 - val_loss: 0.3955 - val_accuracy:
0.8507
Epoch 6/10
117/117 [==============================] - 1s 5ms/step - loss: 0.4282 - accuracy: 0.8291 - val_loss: 0.3912 - val_accuracy:
0.8625
Epoch 7/10
117/117 [==============================] - 1s 5ms/step - loss: 0.4175 - accuracy: 0.8342 - val_loss: 0.3799 - val_accuracy:
0.8604
Epoch 8/10
117/117 [==============================] - 1s 6ms/step - loss: 0.4105 - accuracy: 0.8358 - val_loss: 0.3743 - val_accuracy:
0.8636
Epoch 9/10
117/117 [==============================] - 1s 6ms/step - loss: 0.4022 - accuracy: 0.8401 - val_loss: 0.3685 - val_accuracy:
0.8679
Epoch 10/10
117/117 [==============================] - 1s 5ms/step - loss: 0.3973 - accuracy: 0.8434 - val_loss: 0.3625 - val_accuracy:
0.8700
30/30 [==============================] - 0s 4ms/step - loss: 0.3625 - accuracy: 0.8700
Loss: 0.36245355010032654, Accuracy: 0.8700322508811951
```

EXPERIMENT- 9:

Implement word embeddings for IMDB dataset.

Aim: Implement word embeddings for IMDB dataset.

PROGRAM:

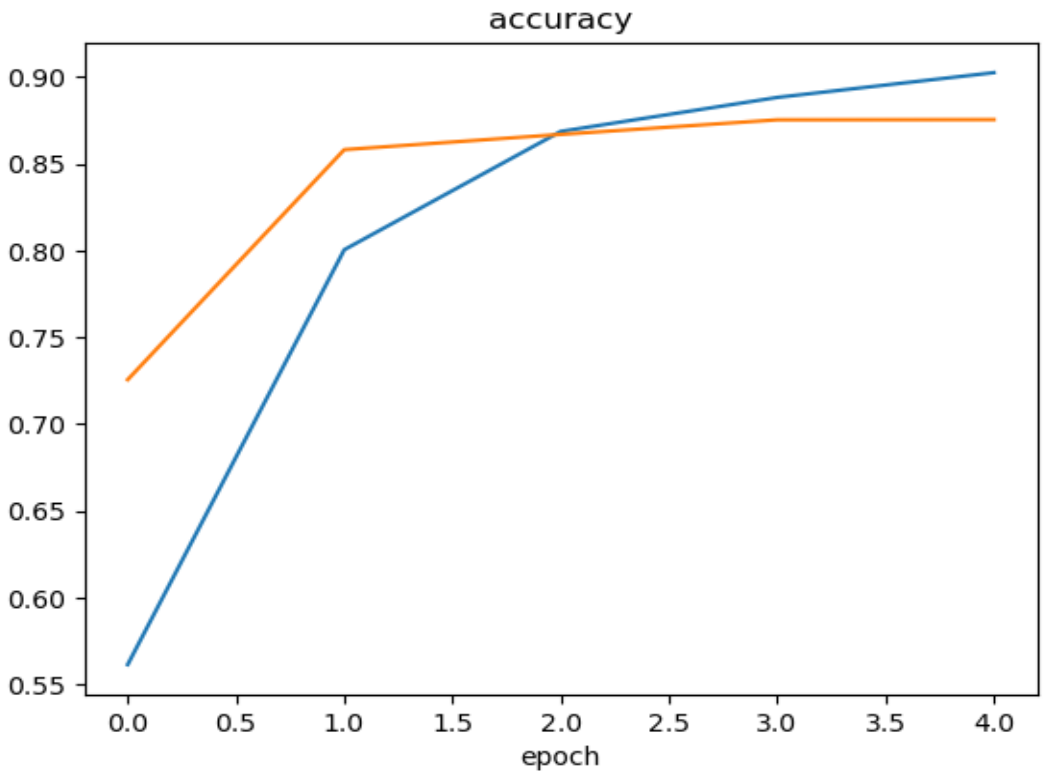
```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from matplotlib import pyplot as plt
# Set random seeds for reproducibility
np.random.seed(0xdeadbeef)
tf.random.set_seed(0xdeadbeef)
# Load the IMDB dataset
imdb = keras.datasets.imdb
num_words = 20000
(train_data, train_labels), (test_data, test_labels) = imdb.load_data(num_words=num_words)
# Preprocess the vocabulary
vocabulary = imdb.get_word_index()
vocabulary = {k:(v+3) for k,v in vocabulary.items()}
vocabulary["<PAD>"] = 0
vocabulary["<START>"] = 1
vocabulary["<UNK>"] = 2
vocabulary["<UNUSED>"] = 3
index = {v: k for k, v in vocabulary.items()}
# Decode a review
def decode_review(text):
    return ' '.join([index.get(i, '?') for i in text])
# Pad sequences
maxlen = 256
```

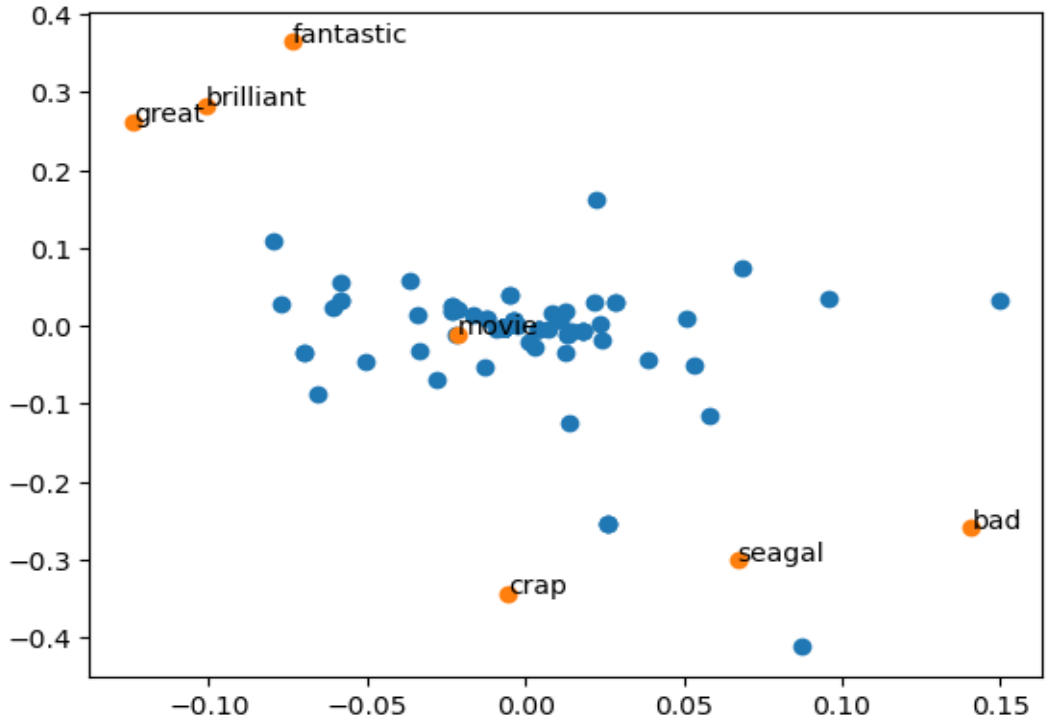
```
train_data = keras.preprocessing.sequence.pad_sequences(train_data,
value=vocabulary["<PAD>"], padding='post', maxlen=maxlen)

test_data = keras.preprocessing.sequence.pad_sequences(test_data,
value=vocabulary["<PAD>"], padding='post', maxlen=maxlen)

# Build the model

model = keras.Sequential([

    keras.layers.Embedding(len(vocabulary), 2, input_length=maxlen),

    keras.layers.Flatten(),

    keras.layers.Dropout(0.5),

    keras.layers.Dense(5),

    keras.layers.Dense(1, activation='sigmoid')

])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model

history = model.fit(train_data, train_labels, epochs=5, batch_size=100,
validation_data=(test_data, test_labels), verbose=1)

# Plot accuracy

def plot_accuracy(history, miny=None):

    acc = history.history['accuracy']

    test_acc = history.history['val_accuracy']

    epochs = range(len(acc))

    plt.plot(epochs, acc)

    plt.plot(epochs, test_acc)

    if miny:

        plt.ylim(miny, 1.0)

    plt.title('accuracy')

    plt.xlabel('epoch')

    plt.show()

plot_accuracy(history)

# Plot word embeddings
```

```
get_embed_out = keras.backend.function([model.layers[0].input], [model.layers[1].output])
layer_output = get_embed_out([test_data[0]])
words = layer_output[0]
plt.scatter(words[:,0], words[:,1])
review = ['great', 'brilliant','crap','bad', 'fantastic', 'movie', 'seagal']
enc_review = tf.constant([vocabulary[word] for word in review])
words = get_embed_out([enc_review])[0]
plt.scatter(words[:,0], words[:,1])
for i, txt in enumerate(review):
    plt.annotate(txt, (words[i,0], words[i,1]))
plt.show()
```

OUTPUT:

```
Epoch 1/5
250/250 [==============================] - 2s 5ms/step - loss: 0.6804 - accuracy: 0.5613 - val_loss: 0.6102 - val_accuracy:
0.7256
Epoch 2/5
250/250 [==============================] - 1s 5ms/step - loss: 0.4573 - accuracy: 0.8005 - val_loss: 0.3581 - val_accuracy:
0.8582
Epoch 3/5
250/250 [==============================] - 1s 5ms/step - loss: 0.3234 - accuracy: 0.8687 - val_loss: 0.3226 - val_accuracy:
0.8670
Epoch 4/5
250/250 [==============================] - 1s 5ms/step - loss: 0.2715 - accuracy: 0.8883 - val_loss: 0.3006 - val_accuracy:
0.8754
Epoch 5/5
250/250 [==============================] - 1s 5ms/step - loss: 0.2380 - accuracy: 0.9026 - val_loss: 0.2960 - val_accuracy:
0.8755
```


accuracy
0.90
0.85
0.80
0.75
0.70
0.65
0.60
0.55
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
epoch


fantastic
brilliant
great
movie
bad
seagal
crap
0.4
0.3
0.2
0.1
0.0
−0.1
−0.2
−0.3
−0.4
−0.10
−0.05
0.00
0.05
0.10
0.15

EXPERIMENT- 10:

Implement a Recurrent Neural Network for IMDB movie review classification problem.

Aim: Implement a Recurrent Neural Network for IMDB movie review classification problem.

PROGRAM:

```
from keras.models import Sequential

from keras.layers import Embedding, Dense, SimpleRNN

from keras.datasets import imdb

from keras.preprocessing import sequence

import matplotlib.pyplot as plt

max_word_range = 10000

# Loading the IMDB dataset

(input_train, output_train), (input_test, output_test) =
imdb.load_data(num_words=max_word_range)

print(len(input_train), 'train sequences.')

print(len(input_test), 'test sequences.')

max_word_amount = 500

# Padding sequences to ensure uniform input size

input_train = sequence.pad_sequences(input_train, maxlen=max_word_amount)

input_test = sequence.pad_sequences(input_test, maxlen=max_word_amount)

print('input_train shape:', input_train.shape)

print('input_test shape:', input_test.shape)

# Building the RNN model

model = Sequential([

 Embedding(max_word_range, 32, input_length=max_word_amount),

 SimpleRNN(32), # Using SimpleRNN for the recurrent layer
```

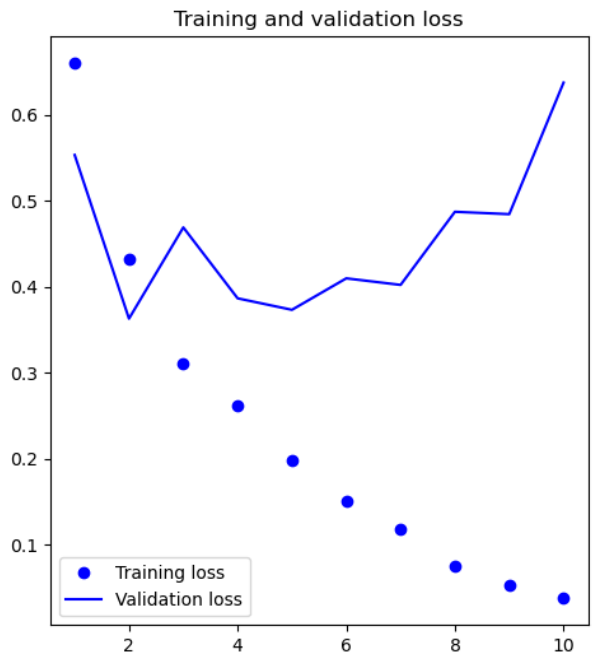
```
 Dense(1, activation='sigmoid')
])
model.summary()
# Compiling the model
model.compile(optimizer='rmsprop',
 loss='binary_crossentropy',
 metrics=['accuracy'])
# Training the model
history = model.fit(input_train, output_train,
 epochs=10,
 batch_size=128,
 validation_split=0.2)
# Extracting loss and accuracy from the history object
loss = history.history['loss']
acc = history.history['accuracy'] # Updated from 'acc' to 'accuracy' based on newer Keras
versions
val_loss = history.history['val_loss']
val_acc = history.history['val_accuracy'] # Updated from 'val_acc' to 'val_accuracy'
epochs = range(1, len(loss) + 1)
# Plotting training and validation loss and accuracy
f, axarr = plt.subplots(1, 2, figsize=(12, 6))
# Loss plot
axarr[0].plot(epochs, loss, 'bo', label='Training loss')
axarr[0].plot(epochs, val_loss, 'b', label='Validation loss')
axarr[0].set_title('Training and validation loss')
axarr[0].legend()
# Accuracy plot
axarr[1].plot(epochs, acc, 'bo', label='Training accuracy')
```

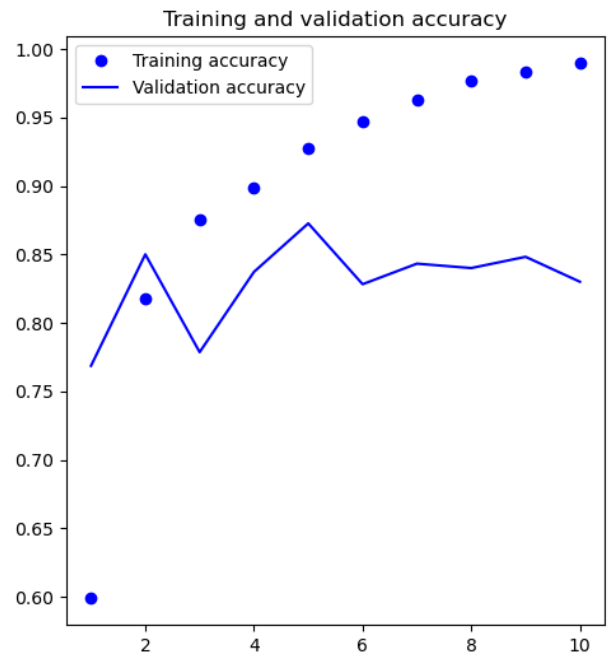
```
axarr[1].plot(epochs, val_acc, 'b', label='Validation accuracy')
axarr[1].set_title('Training and validation accuracy')
axarr[1].legend()
plt.show()
# Evaluating the model on the test set
score = model.evaluate(input_test, output_test)
print('Test Loss:', score[0])
print('Test Accuracy:', score[1])
# Making predictions
predictions = model.predict(input_test[0:5])
true_labels = output_test[0:5]
print("0 is a negative comment, 1 is a positive comment.")
for i in range(len(predictions)):
 print("Prediction:", int(round(predictions[i][0])), ", True Label:", true_labels[i])
```

OUTPUT:

```
WARNING:tensorflow:From C:\Users\KOTES\anaconda3\lib\site-packages\keras\src\losses.py:2976: The name tf.losses.sparse_softm
ax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

25000 train sequences.
25000 test sequences.
input_train shape: (25000, 500)
input_test shape: (25000, 500)
WARNING:tensorflow:From C:\Users\KOTES\anaconda3\lib\site-packages\keras\src\backend.py:873: The name tf.get_default_graph i
s deprecated. Please use tf.compat.v1.get_default_graph instead.

Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 embedding (Embedding)       (None, 500, 32)           320000

 simple_rnn (SimpleRNN)      (None, 32)                2080

 dense (Dense)               (None, 1)                 33

=================================================================
Total params: 322113 (1.23 MB)
Trainable params: 322113 (1.23 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
WARNING:tensorflow:From C:\Users\KOTES\anaconda3\lib\site-packages\keras\src\optimizers\__init__.py:309: The name tf.train.O
ptimizer is deprecated. Please use tf.compat.v1.train.Optimizer instead.

Epoch 1/10
WARNING:tensorflow:From C:\Users\KOTES\anaconda3\lib\site-packages\keras\src\utils\tf_utils.py:492: The name tf.ragged.Ragge
dTensorValue is deprecated. Please use tf.compat.v1.ragged.RaggedTensorValue instead.

WARNING:tensorflow:From C:\Users\KOTES\anaconda3\lib\site-packages\keras\src\engine\base_layer_utils.py:384: The name tf.exe
cuting_eagerly_outside_functions is deprecated. Please use tf.compat.v1.executing_eagerly_outside_functions instead.
```

```
_________________________________________________________________
Epoch 1/10
157/157 [==============================] - 19s 113ms/step - loss: 0.6234 - accuracy: 0.6407 - val_loss: 0.4538 - val_accurac
y: 0.7996
Epoch 2/10
157/157 [==============================] - 18s 114ms/step - loss: 0.3955 - accuracy: 0.8316 - val_loss: 0.4580 - val_accurac
y: 0.8032
Epoch 3/10
157/157 [==============================] - 19s 121ms/step - loss: 0.3415 - accuracy: 0.8615 - val_loss: 0.3521 - val_accurac
y: 0.8504
Epoch 4/10
157/157 [==============================] - 19s 123ms/step - loss: 0.2553 - accuracy: 0.9035 - val_loss: 0.4546 - val_accurac
y: 0.7886
Epoch 5/10
157/157 [==============================] - 20s 126ms/step - loss: 0.2011 - accuracy: 0.9254 - val_loss: 0.4097 - val_accurac
y: 0.8504
Epoch 6/10
157/157 [==============================] - 16s 103ms/step - loss: 0.1526 - accuracy: 0.9455 - val_loss: 0.4188 - val_accurac
y: 0.8266
Epoch 7/10
157/157 [==============================] - 16s 104ms/step - loss: 0.1169 - accuracy: 0.9601 - val_loss: 0.4582 - val_accurac
y: 0.8322
Epoch 8/10
157/157 [==============================] - 18s 116ms/step - loss: 0.0793 - accuracy: 0.9747 - val_loss: 0.4741 - val_accurac
y: 0.8594
Epoch 9/10
157/157 [==============================] - 19s 118ms/step - loss: 0.0594 - accuracy: 0.9829 - val_loss: 0.7399 - val_accurac
y: 0.7510
Epoch 10/10
157/157 [==============================] - ETA: 0s - loss: 0.0465 - accuracy: 0.9862
```






```
782/782 [==============================] - 21s 27ms/step - loss: 0.6144 - accuracy: 0.8209
Test Loss: 0.6144405603408813
Test Accuracy: 0.8209199905395508
1/1 [==============================] - 0s 227ms/step
0 is a negative comment, 1 is a positive comment.
Prediction: 0 , True Label: 0
Prediction: 1 , True Label: 1
Prediction: 0 , True Label: 1
Prediction: 1 , True Label: 0
Prediction: 1 , True Label: 1
```